

Latency-Fairness vs. Throughput Tradeoffs in Non-Preemptive Shared GPU Scheduling

Jianing Ou
jou32@wisc.edu
University of Wisconsin-Madison

Yayen Lin
lin383@wisc.edu
University of Wisconsin-Madison

Xuechun Jin
xjin252@wisc.edu
University of Wisconsin-Madison

Matthew Kearney
mtkearney@wisc.edu
University of Wisconsin-Madison

Abstract

Modern cloud GPU platforms increasingly share hardware across multiple concurrent workloads, from latency-sensitive inference to throughput-oriented training. Because GPUs are fundamentally non-preemptive (a kernel cannot be interrupted once it starts), scheduling decisions critically impact job waiting times, fairness, and throughput. Understanding these tradeoffs is essential for designing effective shared GPU systems.

We study the tradeoffs among latency, fairness, and throughput in non-preemptive GPU scheduling through comprehensive empirical evaluation. We use ISCAS-85 circuit benchmarks to generate realistic task graphs with diverse dependency structures, mapping each circuit gate to a synthetic CUDA kernel that emulates specific computational characteristics (compute-bound, memory-bound, latency-sensitive) while preserving realistic dependencies. We implement and evaluate six scheduling policies: FIFO, priority-based, and four dependency-aware variants (HighFanout, CriticalPath, LevelAware, Hybrid) across sequential batch execution (launch batches, synchronously wait for batch completion) and concurrent slot-based execution (maintain up to N tasks in flight, asynchronously poll for completions). In the experiment, we measure latency, throughput, GPU utilization, and fairness across balanced and imbalanced workloads with varying batch sizes (32, 128, 512).

Our results ...

1 Introduction

Running only a single application exclusively on a GPU is no longer sufficient for the modern society. Nowadays, one GPU needs to handle both latency-sensitive tasks, such as real-time voice queries, and large-scale computation tasks that occupy the GPU for a long time. Since GPUs are non-preemptive, which means a task cannot be interrupted once it starts running, GPU scheduling becomes an important problem[2]. How to ensure fast response times for latency-sensitive tasks without letting computation-heavy tasks to wait too long becomes a main challenge. Therefore, in this project, we study how different scheduling policies and execution modes affect the trade-offs among latency, throughput, and fairness on a non-preemptive GPU.

In our experiment, we use a Directed Acyclic Graph (DAG) to represent the relationships between tasks. In the graph, each node represents a task, and each edge represents a dependency between two tasks. A node can only execute after all its predecessor nodes have completed.

we did...

results...

2 Experiment Setup

Hardware Configuration. Our experiments are conducted on a Google Colab T4 GPU.

Benchmark. We use circuit benchmark files provided by the course staff via the LSIM framework[1]. It contains 11 circuits in total and has three sizes: Small (c17, c432, c499, c880), Medium (c1355, c1908, c2670, c3540), and Large (c5315, c6288, c7552).

Measurement.

3 Methodology

4 Results

5 Discussion

6 Future Work

7 Conclusion

References

- [1] Course Staff. 2026. LSIM Framework and ISCAS-85 Circuit Benchmarks. Course-provided benchmark infrastructure.
- [2] Shinpei Kato, Karthik Lakshmanan, Ragunathan Rajkumar, and Yutaka Ishikawa. 2011. TimeGraph: GPU Scheduling for Real-Time Multi-Tasking Environments. In *Proceedings of the 2011 USENIX Annual Technical Conference*.